

GUÍA PASO A PASO

PyAssistant Analytics

Dashboard personal de productividad con IA · Built by Iván Arias

Índice

#

Sección

Tiempo estimado

1

Requisitos previos

5 min

2

Instalación local

15 min

3

Configuración de IA (Gemini)

5 min

4

Generar datos de ejemplo

2 min

5

Ejecutar el servidor

2 min

6

Probar la app localmente

10 min

7

Entender la estructura del proyecto

15 min

8

Deploy en Railway

20 min

9

Publicar en LinkedIn

15 min

10

Troubleshooting

—

1. Requisitos Previos

Antes de empezar, asegúrate de tener instalado:

Herramienta

Versión mínima

Cómo verificar

Python

3.11+

`python --version`

pip

21+

`pip --version`

Git

2.30+

`git --version`

Navegador moderno

Chrome/Firefox/Edge

—

Editor de código

VSCode recomendado

—

 **Tip:** Si no tienes Python, descárgalo de python.org. Marca la opción "Add to PATH" durante la instalación.

Cuentas opcionales (pero recomendadas):

2. Instalación Local

Paso 1: Abrir terminal en la carpeta del proyecto

Navega a la carpeta donde está el proyecto. En Windows, abre la terminal en la carpeta raíz del proyecto.

Paso 2: Crear entorno virtual de Python

Un entorno virtual aísla las dependencias de tu proyecto.

```
bash# Windows (PowerShell o CMD)python -m venv venv# Activar el entornovenv\Scripts\activate#  
Verás (venv) al inicio de tu terminal, eso significa que está activado
```

 **Importante:** Cada vez que abras una nueva terminal, debes activar el entorno virtual antes de ejecutar comandos del proyecto.

Paso 3: Instalar dependencias

Las dependencias están listadas en `backend/requirements.txt`

```
bash# Asegúrate de estar en la carpeta backendcd backend# Instalar todas las dependenciaspip  
install -r requirements.txt# Esto instalará: fastapi, sqlalchemy, pydantic, google-genai, etc.
```

Paso 4: Verificar instalación

```
bash# Verificar que todo se instaló correctamente
pip list# Deberías ver: fastapi, sqlalchemy,
pydantic, google-genai, uvicorn, etc.
```

3. Configuración de IA (Gemini)

La IA es opcional pero recomendada. Sin API key, el dashboard funciona pero las funciones de IA estarán deshabilitadas.

Paso 1: Obtener API key gratuita

Paso 2: Configurar variable de entorno

```
bash# En la carpeta backend# Copiar el archivo de ejemplo
cp .env.example .env# Abrir .env con tu
editor favorito (VSCode, Notepad, etc.)
code .env# Reemplazar la línea:
GEMINI_API_KEY=your_gemini_api_key_here# Con tu API key real:
GEMINI_API_KEY=AIza...tu_key_real...
```

 **Seguridad:** El archivo .env está en .gitignore, NUNCA se sube a GitHub. Mantén tu API key privada.

Paso 3: Verificar configuración

```
bash# La app detectará automáticamente tu key al iniciar# Verás en la consola: "✅ Gemini AI
configured successfully"# En lugar de: "⚠️ GEMINI_API_KEY not configured"
```

4. Generar Datos de Ejemplo

Para que el dashboard tenga datos que mostrar, ejecuta el script de seed:

```
bash# Desde la carpeta backend
python -m data.seed# Esto generará:
# - 8 categorías (Coding, Meetings, Learning, etc.)
# - ~100 actividades distribuidas en los últimos 30 días
# - Con puntuaciones de productividad realistas (1-10)
# - Tiempos variados (30min - 2.5h)
```

Verás un output como:

```
bash 🌱 Seeding database...✅ Created 8 categories✅ Created 100+ activities across 30 days 📊
Summary by category: 💻 Coding: 25 activities, 35.2h 📖 Learning: 18 activities, 22.1h 🤖 AI/ML:
12 activities, 18.5h ... 🎉 Database seeded successfully!
```

5. Ejecutar el Servidor

Paso 1: Iniciar el servidor de desarrollo

```
bash# Desde la carpeta backend
uvicorn main:app --reload# Verás:
# INFO: Started server process#
INFO: Waiting for application startup# ✅ Database initialized#
INFO: Application startup complete#
INFO: Uvicorn running on http://127.0.0.1:8000
```

 **El flag --reload:** Hace que el servidor se reinicie automáticamente cuando edites el código. Quítalo en producción.

Paso 2: Verificar que está corriendo

Abre tu navegador en: <http://127.0.0.1:8000>

Deberías ver el dashboard con:

6. Probar la Aplicación

Una vez que el dashboard carga, prueba lo siguiente:

Paso 1: Verificar el dashboard general

Paso 2: Probar gráficos

Paso 3: Probar el chat con IA

Paso 4: Agregar una actividad nueva

Paso 5: Probar la API directamente

Abre <http://127.0.0.1:8000/docs> para ver la documentación interactiva de la API (Swagger UI)

Ahí puedes probar todos los endpoints:

7. Entender la Estructura del Proyecto

Esta es la estructura completa del proyecto:

```
textpyassistant-analytics/└─ backend/                                # Código Python (FastAPI)| └─ main.py
# Punto de entrada de la app| └─ config.py                            # Configuración (lee .env)| └─
database.py                # Conexión a SQLite| └─ requirements.txt    # Dependencias Python|
└─ .env.example            # Plantilla de variables de entorno| || └─ models/                                #
Modelos de base de datos (SQLAlchemy)| | └─ activity.py              # Tabla de actividades| |
└─ category.py            # Tabla de categorías| || └─ schemas/                # Validación de
datos (Pydantic)| | └─ activity.py| | └─ category.py| || └─ api/                    #
Endpoints REST| | └─ activities.py    # CRUD de actividades| | └─ analytics.py    #
Agregaciones y estadísticas| | └─ chat.py                # Endpoints de IA| || └─ ai/
# Integración con Gemini| | └─ gemini.py                  # Cliente de Google Gemini| || └─ tests/
# Tests automatizados| | └─ test_api.py                  # Tests de endpoints| || └─ data/
# Scripts auxiliares| | └─ seed.py                        # Genera datos de ejemplo|└─ frontend/
# Interfaz web| | └─ index.html                            # Página principal| └─ css/| | └─ styles.css
# Estilos| | └─ js/| | └─ app.js                            # Lógica del frontend|└─ data/
# Bases de datos| | └─ pyassistant.db                    # SQLite (se crea al ejecutar)|└─ Dockerfile
# Para deploy con Docker|└─ docker-compose.yml          # Orquestación de contenedores|└─
railway.json          # Config de Railway|└─ .gitignore|└─ .dockerignore└─ README.md
# Documentación principal
```

Conceptos clave por capa:

Capa

Tecnología

Responsabilidad

Frontend

HTML + CSS + JS

Interfaz de usuario, visualizaciones

API

FastAPI

Recibir requests, validar, enviar responses

Schemas

Pydantic

Validar estructura de datos

Business Logic

Python

Procesar datos, calcular analytics

AI

google-genai

Generar insights y chat con datos

Models

SQLAlchemy

Mapear objetos Python a tablas SQL

Database

SQLite

Almacenar datos persistente

8. Deploy en Railway (Producción)

Railway te da una URL pública gratis. En 20 minutos tendrás tu dashboard en internet.

Paso 1: Crear repositorio en GitHub

```
bash# En la carpeta raíz del proyecto
cd pyassistant-analytics# Inicializar git
git init
git add .
git commit -m "Initial commit: PyAssistant Analytics v1.0"# Crear repo en github.com (sin inicializar con README)# Luego conectar:
git remote add origin https://github.com/TU_USUARIO/pyassistant-analytics.git
git branch -M main
git push -u origin main
```

Paso 2: Crear cuenta en Railway

Paso 3: Deploy desde GitHub

Paso 4: Configurar variables de entorno

Paso 5: Esperar el deploy

Railway construirá y desplegará la app en 3-5 minutos. Te dará una URL como:

```
https://pyassistant-analytics-production.up.railway.app
```

 **¡Listo!** Tu dashboard ahora es accesible públicamente. Comparte la URL en tu LinkedIn y CV.

Paso 6: Verificar el deploy

9. Publicar en LinkedIn

Una vez deployado, anuncia tu proyecto con este post:

```
text🚀 Acabo de lanzar PyAssistant Analytics – un dashboard personal de productividad con IAStack
técnico:✅ FastAPI + SQLAlchemy + Pydantic✅ Google Gemini AI (análisis y chat)✅ Chart.js para
visualizaciones✅ Docker + Railway (deploy automático)✅ Bilingüe ES/ENCaracterísticas:📊 Tracking
de tiempo por categoría📈 Visualizaciones en tiempo real💬 Chat con IA sobre tus propios datos🌐
Multi-idioma🔗 Deploy en 1-click🔗 Demo en vivo: [TU_URL]💻 Código:
github.com/TU_USUARIO/pyassistant-analytics#Python #FastAPI #Gemini #DataVisualization
#ProductivityHacks #IndieHacker #BuildInPublic
```

10. Troubleshooting (Problemas Comunes)

Si encuentras errores, busca tu problema aquí:

Problema

Causa probable

Solución

ModuleNotFoundError: No module named 'fastapi'

No activaste el entorno virtual o no instalaste deps

Activa venv y ejecuta: `pip install -r requirements.txt`

Port 8000 is already in use

Otra app usa el puerto 8000

Usa otro puerto: `uvicorn main:app --port 8001`

API endpoint returns 503

No configuraste `GEMINI_API_KEY` o es inválida

Verifica tu `.env` y que la key empiece con "Alza"

Database is locked

Otro proceso usa la DB

Cierra otras terminales con la app corriendo

ModuleNotFoundError: No module named 'google.genai'

No instalaste google-genai

`pip install google-genai`

Las gráficas no se muestran

Chart.js no cargó

Verifica tu conexión a internet (Chart.js viene de CDN)

Error en deploy: "Dockerfile not found"

Railway no encuentra el Dockerfile

Verifica que Dockerfile está en la raíz del proyecto

ModuleNotFoundError en Railway

requirements.txt no está completo

Verifica que pip freeze muestra todas las deps necesarias

 **Si nada funciona:** Abre un issue en GitHub con el error completo (con stack trace). La comunidad o yo podemos ayudarte.

Recursos Adicionales

Para profundizar en las tecnologías usadas:

Recurso

Link

Descripción

FastAPI Docs

fastapi.tiangolo.com

Documentación oficial del framework

SQLAlchemy ORM

docs.sqlalchemy.org

ORM usado para base de datos

Google Gemini

ai.google.dev

Docs de la IA usada

Chart.js

chartjs.org

Librería de gráficos del frontend

Railway

docs.railway.app

Plataforma de deploy usada

Jupyter Notebook del proyecto

PyAssistant_Tutorial.ipynb

Tutorial paso a paso del código

¡Éxito con tu proyecto, Iván! 🚀

Para dudas, contacta: ivan.ariasg@hotmail.com